COMMUNITY OVER CODE ASIA 2026 · INCUBATOR TRACK

Build Once, Run on Any Linux

SynxDB CE — a truly portable binary distribution of **Apache Cloudberry (Incubating)**

Xin “Shine” Zhang — Cofounder / CTO, Synx Data Labs

Ed Espino — Cofounder / CEO, Synx Data Labs

BACKGROUND

What is Apache Cloudberry (Incubating)?

Postgres-compatible MPP

Massively-parallel analytics on a PostgreSQL kernel.

Greenplum lineage

Community successor, now an ASF incubating project.

SynxDB CE

Our Community Edition build of it.

The engine isn't the hard part — **shipping it as something you can install is.**

THE GAP

A source tarball isn't something you can run

ASF ships source

Official Cloudberry releases are source tarballs — correct for ASF governance.

Users expect a binary

Hobbyists, evaluators, enterprises: dnf install, apt install, done.

Closing that last mile — *without fighting ASF governance* — is what SynxDB CE does.

THE LEGACY TRAP

The obvious approach is a trap

“Just build one package per distro” — and watch it multiply.

glibc symbols

Symbol versions differ across distros.

Compiler ABIs

libstdc++ / GCC versions diverge.

Bundled libs

OpenSSL, Perl, Python drift over time.

RPM vs DEB

Two packaging formats, two ways.

→ a dozen variants of the **same release** to build, test, and maintain.

THE COST

The matrix that explodes

Every cell is a build to produce, a CI lane to keep green, and a customer to support.

THE BET

Build once, run anywhere glibc is recent enough

One arm64 tree · glibc the only external dependency · no per-distro rebuild.

1 · Bundle toolchain

From-source GCC

2 · glibc 2.28 floor

Build on Rocky 8

3 · glibc-only dep

Vendor via $ORIGIN rpath

4 · RPM + DEB

Both, one container

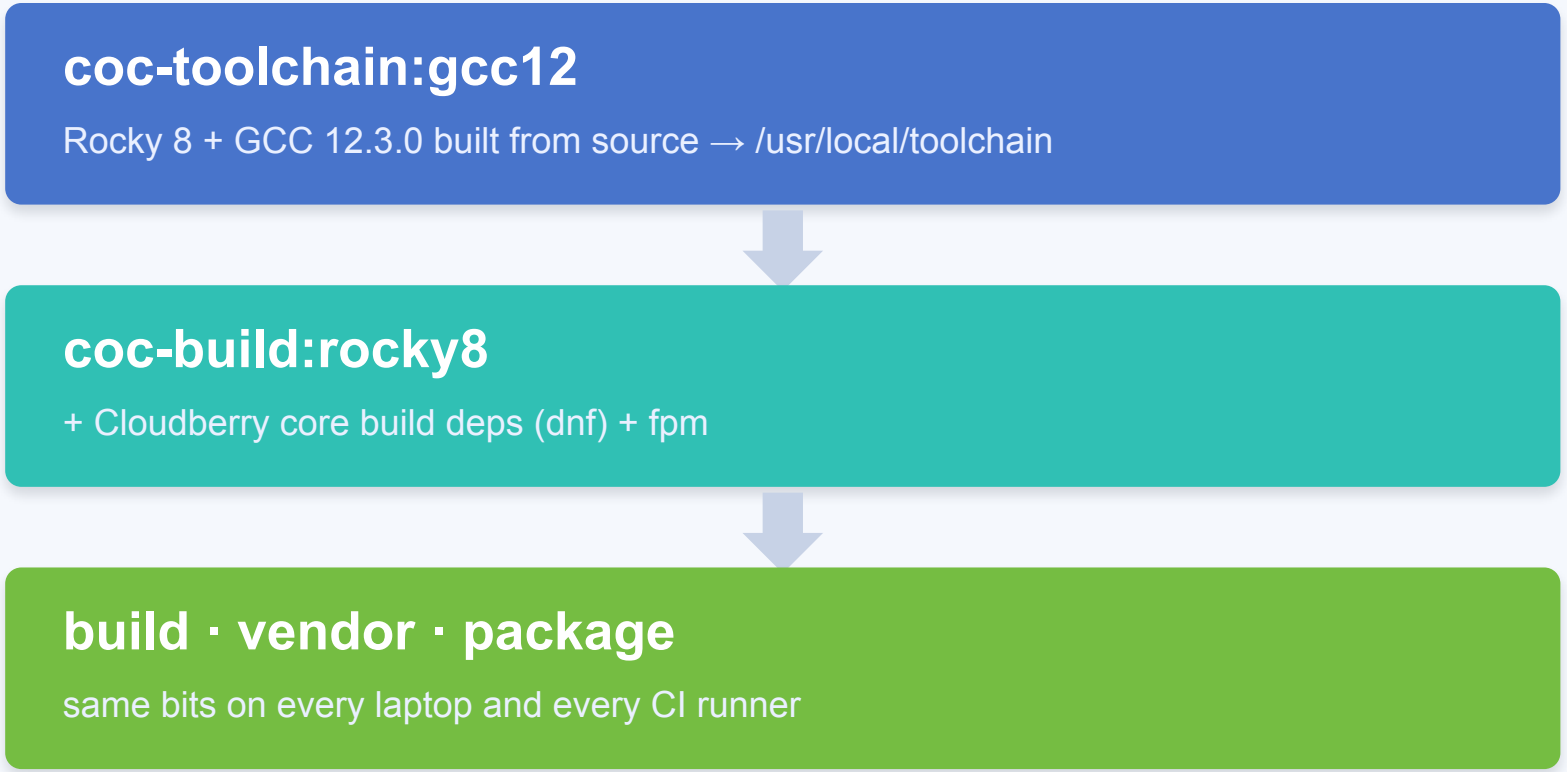
5 · CI gate

ldd-gated per commit

MOVE 1

Bundle the entire toolchain

The host OS contributes nothing to the compile.



MOVE 1B

Statically link the C++ runtime

```
LDFLAGS = -static-libstdc++ -static-libgcc
```

Newer C++ runtime

Our from-source GCC ships a newer libstdc++ / libgcc_s than any target.

Link it in

Static-link them so a clean target doesn't need them.

glibc stays dynamic

The one thing we rely on the host for.

MOVE 2

A glibc 2.28 ABI floor (Rocky 8)

glibc's backward-compatibility guarantee does the rest.

Linked at 2.28

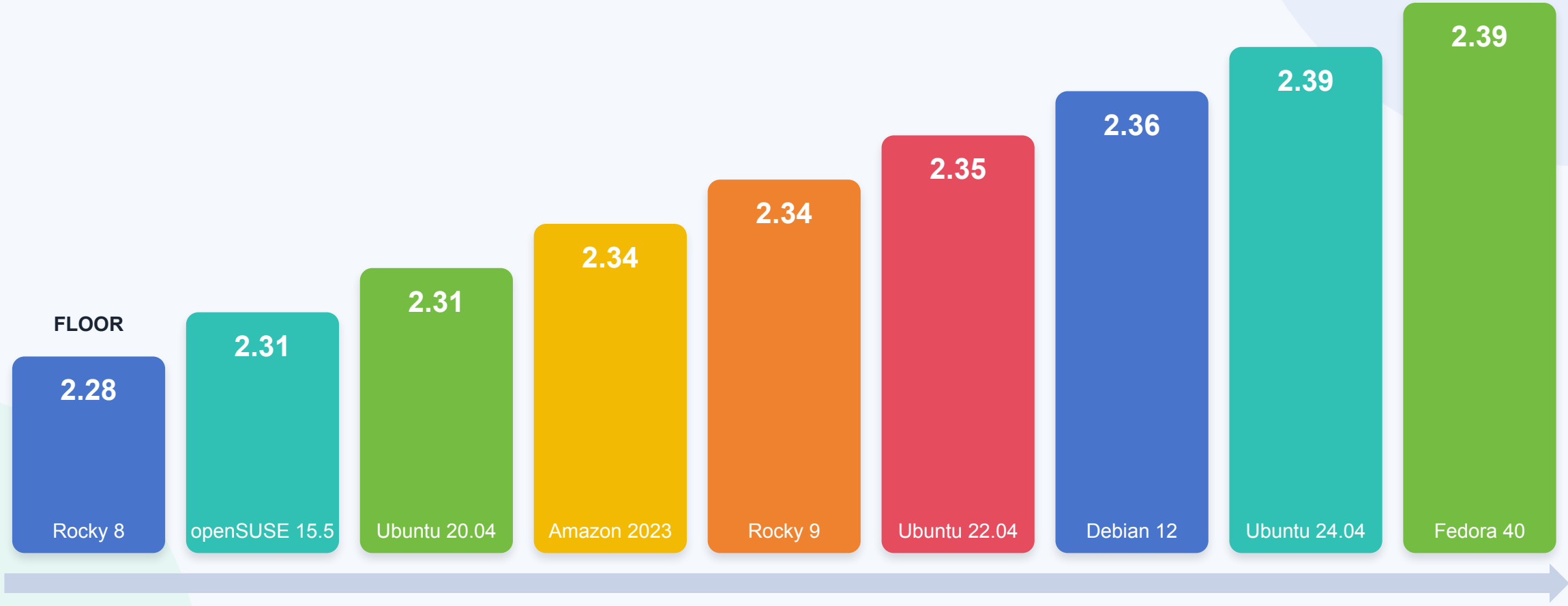
Runs unchanged on any distro with glibc ≥ 2.28.

Target the floor

Pick the oldest glibc you care about — everything newer is free.

REACH

One binary, nine distros



glibc 2.28 → 2.39 · ~6 years of releases · three package managers · one binary

MOVE 3

glibc is the only runtime dependency

Everything else is vendored into lib/ and linked via an $ORIGIN rpath.

OpenSSL · Xerces-C · OpenLDAP · krb5 · APR · libevent · readline · zlib · libxml2

libxslt · libcurl · libuuid · bzip2 · Perl · Python · PAM · libzstd · …

No external package-manager dependencies. *No “install these 40 packages first.”*

MOVE 3B

How the binary finds its own libraries

$ORIGIN rpath, computed per ELF file with patchelf.

bin/postgres → rpath → $ORIGIN/../lib

lib/postgresql/.../some.so → rpath → $ORIGIN/../../.. (relative to its depth)

ldd → every dependency resolves inside the package · glibc is the lone “ask the OS” line

ldd -r: every dep resolves inside the package

27 of 34 libraries load from the bundle — the only 7 from the host are glibc + its loader.

```
$ ldd -r bin/postgres
  libxerces-c-3.2.so  => /usr/local/synxdb-ce/lib/libxerces-c-3.2.so
  libssl.so.1.1       => /usr/local/synxdb-ce/lib/libssl.so.1.1
  libcrypto.so.1.1    => /usr/local/synxdb-ce/lib/libcrypto.so.1.1
  libgssapi_krb5.so.2 => /usr/local/synxdb-ce/lib/libgssapi_krb5.so.2
  … 23 more           => /usr/local/synxdb-ce/lib/        # 27 vendored — all inside the
package
  libpthread.so.0     => /lib/aarch64-linux-gnu/libpthread.so.0
  libm.so.6           => /lib/aarch64-linux-gnu/libm.so.6
  libc.so.6           => /lib/aarch64-linux-gnu/libc.so.6
  /lib/ld-linux-aarch64.so.1      # + libdl, librt, libresolv  →  7 from glibc
# -r → relocations processed · 0 unresolved symbols
```

PACKAGING

Meet fpm — one tool, every package format

“Effing Package Management” — one build tree in, RPM + DEB out.

```
$ fpm -s dir -t rpm  -n synxdb-ce -v 2.1.0 -a arm64 \
      --prefix /usr/local/synxdb-ce  -C /usr/local/synxdb-ce  .
$ fpm -s dir -t deb  -n synxdb-ce -v 2.1.0 -a arm64  …   # same tree, same flags
```

Source: a dir

-s dir packages the staged tree as-is.

Target: rpm or deb

-t rpm / -t deb from identical inputs.

No spec files

Metadata on the CLI — no .spec, no control.

MOVE 4

One tree → RPM and DEB

Single build tree

/usr/local/synxdb-ce — one source of truth.

Both via fpm

RPM and DEB from the same container.

~39 MB per package

The whole compiler-built, fully-vendored core.

“Fully bundled” usually means gigabytes. **Here it's 39 MB.**

MOVE 5

Portability as a CI gate

We don't hope it's portable — we enforce it on every commit.

ldd scan

Fail if anything is “not found” beyond glibc.

postgres --version

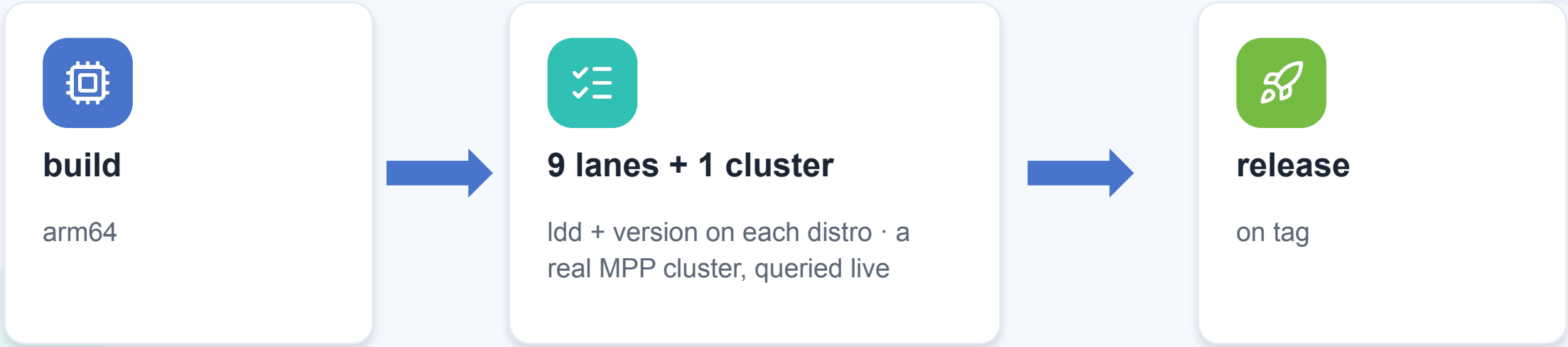
On a clean install of each target distro.

All 9 distros

Fail-fast off — one flaky lane can't mask the rest.

PROOF

The CI fan-out: prove it two ways



11 jobs per commit · 1 build + 9 portability lanes + 1 cluster

LIVE DEMO

A real MPP cluster from the portable binary

```
$ make cluster
```

Coordinator | **Standby** | **Primary ×2** | **Mirror ×2**

```
SELECT * FROM gp_segment_configuration;   -- all segments up
```

TRADE-OFFS

Why not just ship a container?

	Container image	Portable package
Bare-metal / on-prem	needs a runtime + privileges	**native install**
Kernel / cgroup config	inherits host complexity	**none**
Size	still large (full userland)	**~39 MB**
Ops familiarity	“another thing to run”	**dnf / apt they know**

We chose the **package.**

WHAT DIDN'T WORK · 1

The RPM build-id collision

1 · Symptom → 2 · Root cause → 3 · Fix

The RPM won't install — it refuses to co-exist with base-OS packages.

```
$ dnf install synxdb-ce-2.1.0-1.aarch64.rpm
Transaction test error:
  file /usr/lib/.build-id/3f/2a9c… conflicts between
    synxdb-ce-2.1.0   and   pcre2-10.40, libselinux-3.4, …
✗ RPM refuses to install
```

WHAT DIDN'T WORK · 1

The RPM build-id collision

1 · Symptom → **2 · Root cause** → 3 · Fix

rpmbuild auto-creates /usr/lib/.build-id/* symlinks keyed on each ELF's GNU build-id — and our vendored copies share build-ids with the base OS packages.

```
$ readelf -n lib/libpcre2-8.so.0 | grep 'Build ID'
  Build ID: 3f2a9c…   ← our vendored copy (byte-identical to the floor's)
  Build ID: 3f2a9c…   ← /usr/lib64/libpcre2-8.so.0, owned by pkg 'pcre2'
```

WHAT DIDN'T WORK · 1

The RPM build-id collision

1 · Symptom → 2 · Root cause → **3 · Fix**

Tell fpm to skip the build-id links — they point outside our --prefix anyway.

```
fpm -t rpm …  --rpm-rpmbuild-define '_build_id_links none'
✓ installs clean on all 9 distros
# DEB never needed it
```

The vendored lib that broke `su`

1 · Symptom → 2 · Root cause → 3 · Fix

After install, the host's own commands start failing — su, ssh, sudo.

```
$ su - gpadmin
su: cannot open session: Module is unknown
$ ssh node2 hostname
/usr/bin/ssh: libselinux.so.1: no version information available
```

The vendored lib that broke `su`

1 · Symptom → **2 · Root cause** → 3 · Fix

cloudberry-env.sh — the env script our package ships — exports LD_LIBRARY_PATH=$GPHOME/lib, and it leaked into every command the login shell spawned.

```
# cloudberry-env.sh  (shipped in the package, sourced at login)
export LD_LIBRARY_PATH=$GPHOME/lib     ← leaks into su, ssh, sudo …
# host /bin/su then loads OUR libselinux (built on Rocky 8), not the host's
```

WHAT DIDN'T WORK · 2

The vendored lib that broke `su`

1 · Symptom → 2 · Root cause → **3 · Fix**

Confine cloudberry-env.sh to a subshell / su -c — never let it leak into the host shell. The $ORIGIN rpath (Move 3) means the server finds its libs anyway.

```
# source the env only where it's needed, in a subshell — no host leak
( source cloudberry-env.sh; gpdemo … )
$ su - gpadmin -c 'source cloudberry-env.sh; initdb -D /tmp/demo'
✓ host su/ssh keep host libs · server resolves its own via $ORIGIN rpath
```

ROADMAP

What's next

More platforms

x86_64, and a real SUSE install-and-smoke lane.

Symbol ABI gate

objdump -T | grep GLIBC_ in CI, not just ldd.

Supply chain

SBOM, GPG-signed RPM/DEB, SLSA provenance.

Licenses

Build-time scan + bundled NOTICE/LICENSE.

TAKEAWAYS

If you remember one thing…

Build once

A glibc floor + bundled toolchain beats a per-distro matrix.

Remove the host

A 2-layer toolchain takes the host OS out of the build.

glibc-only

Vendor everything else via an $ORIGIN rpath.

Enforce in CI

One tree → RPM + DEB, portability gated.

Cool facts: ~39 MB · 9 distros (glibc 2.28→2.39) · 11 CI jobs/commit · ~430 lines · open-source

GET STARTED

Try it · contribute

```
$ make dist        # build the portable RPM + DEB
$ make smoke       # run the 9-distro portability gate
$ make cluster     # stand up the live MPP demo
```

The repo

github.com/Synx-Data-Labs/2026-cfp-coc-asia — Apache-2.0, the whole pipeline.

Contribute

Upstream to Apache Cloudberry (Incubating) · try SynxDB CE.

REQUIRED NOTICE

Apache Incubator disclaimer

Apache Cloudberry is an effort undergoing incubation at The Apache Software Foundation (ASF), sponsored by the Apache Incubator. Incubation is required of all newly accepted projects until a further review indicates that the infrastructure, communications, and decision-making process have stabilized in a manner consistent with other successful ASF projects.

THANK YOU

Questions?

github.com/Synx-Data-Labs/2026-cfp-coc-asia

Try SynxDB CE · contribute upstream to Apache Cloudberry (Incubating)